

IEEE RO-MAN 2026 · ORAL PRESENTATION

Ontology-Grounded LLM Planning for Robust Table-Clearing Robot Manipulation

Planning over a grounded world model instead of free text

Satuk Buğrahan ÖZTÜRK¹ · Ahmet COŞKUNÇAY¹ · Erhan ÖZTOP^{2,3} · Barış ÖZYER¹

¹ Department of Computer Engineering, Atatürk University, Erzurum, Türkiye

² Department of Artificial Intelligence, Özyeğin University, İstanbul, Türkiye

³ OTRI/SISReC, The University of Osaka, Japan

A simple instruction, a hard execution

“clear the table”

Natural language is a practical interface for human–robot interaction: the user states the goal, not the steps.

But the same sentence maps to a different action sequence in every physical scene.

1

Which objects are actually on the table right now?

Object set changes between and during trials.

2

Is any object blocked or stacked on another?

Support relations decide what can be picked first.

3

Does the scene still match the instruction?

The world moves while the plan is being executed.

A plan that is plausible at the text level can still fail in the actual environment.

Why text-only LLM planning is brittle

1

Static scene description

The world state is written into the prompt once and then stays outside the execution loop. It is already stale by the second action.

2

No feasibility check

Nothing prevents the planner from proposing an action that conflicts with the physical preconditions of the scene.

3

No recovery path

One failed pick or one misplaced object invalidates the remainder of a long-horizon plan.

Prior grounding work helps — **scene graphs, semantic maps and knowledge graphs give the planner explicit objects and relations [4]–[9]** — but these representations are often only loosely connected to execution.

Three lines of work — and the gap between them

LLM task planning

- Instruction → symbolic action sequence
- Long-horizon and service-robot planning
- Corrective reasoning and feedback

[1]–[3], [10]–[15]

Vision-language grounding

- Open-vocabulary 3D scene graphs
- Scene-graph based task planning
- Symbolic inputs derived from vision

[4]–[6], [19]–[26]

Knowledge-augmented planning

- Object ontology knowledge for planning
- Knowledge graphs for decomposition
- Neuro-symbolic robotics

[7]–[9], [27]–[30]

The gap

A task state that persists through manipulation — queried again when a skill fails, not only when the first plan is generated.

What this paper contributes

1

Ontology-grounded state representation

Planning runs over a state built from ontology queries and simulator observations — never over free text alone.

2

Symbolic plan validation with bounded repair

Every plan is checked against permitted actions, known objects and the robot holding state before a single motor command is issued.

3

Failure-triggered replanning

When a skill reports failure, the state is reconstructed from ontology and simulator, and only the remaining task is replanned.

The contribution is the system-level integration of these three parts — evaluated end-to-end against a text-only baseline in PyBullet.

Architecture: from instruction to grounded execution



Ontology as an actively maintained symbolic memory

Object types

Tableware · Cutlery · Containers · Food · Furniture

Pose properties

hasPosition (x, y, z) · hasOrientation (quaternion)

Support relations

onTopOf — is it on the table, stacked, or blocked?

Storage targets

Each semantic category maps to a default destination

EXAMPLE ASSERTIONS

```
plate1    rdf:type    Tableware
```

```
fork1     rdf:type    Cutlery
```

```
apple     rdf:type    Food
```

```
plate1    hasPosition  [xp, yp, zp]
```

```
plate1    hasOrientation [qx, qy, qz, qw]
```

```
apple     onTopOf     fruit_bowl
```

After each action the manipulated object's pose and support relations are rewritten — the ontology is part of the execution loop, not a static description of the initial scene.

Planning over a state, not over a paragraph

$$S_t = \{ O_t, R_t, H_t, L_t \}$$

$$o_i = \{ c_i, p_i, q_i, \ell_i, s_i, b_i \}$$

per-object symbolic record

O_t

Task-relevant objects

Table-related items only — furniture and unrelated entities are excluded.

R_t

Symbolic relations

Object-to-object and object-to-surface dependencies that gate manipulation.

H_t

Robot holding state

Flags an active grasp, so pick / place order stays consistent.

L_t

Available targets

Ontology-derived storage destinations for each category.

The complete ontology is never sent to the LLM. Only task-relevant knowledge is retained, which keeps the prompt small and prevents the planner from proposing actions that conflict with the physical scene.

Symbolic planning with a hard validity gate

PLANNING FUNCTION

$$\pi_t = \text{F_LLM}(\mathbf{x}, S_t, \mathcal{A})$$

$\mathcal{A} = \{\text{pick}, \text{place}, \text{move_home}\}$ — the planner stays purely symbolic; the prompt carries the instruction, the scene, the permitted actions and a required JSON schema.

VALIDATION FUNCTION

$$V(\pi_t, S_t) \in \{0, 1\}$$

Invalid plans are returned to the LLM together with the validation errors, for at most two repair attempts.

Non-empty plan

and only permitted actions from $\mathcal{A}$

Known objects and targets

every referent must exist in S_t

Holding-state consistency

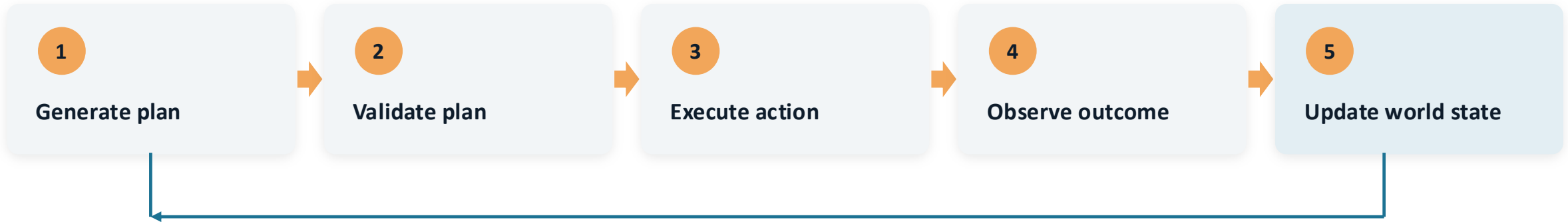
no pick while already holding an object

Correct place semantics

no place with an empty hand or the wrong object

Every validated symbolic action is mapped to a predefined manipulation skill — high-level reasoning stays decoupled from low-level control.

Failure is a trigger, not a dead end



ON SKILL FAILURE

- Reconstruct S_t from the latest ontology and simulator information
- Replan only the remaining task, from the updated state
- No separate external cue is given to the LLM

pick fails when

the object is unavailable, the robot already holds another object, an onTopOf relation blocks it, or grasp verification fails

place fails when

the hand is empty, the wrong object is held, the target is unknown, or the placement check fails

Ontology–simulator differences are recorded, but they do not independently trigger replanning.

Table clearing in PyBullet with a Kuka LBR iiwa

Simulator

PyBullet — dining table, storage area,
household objects

Robot

Kuka LBR iiwa manipulator

LLM backend

Qwen2.5 3B, served locally with Ollama

Decoding

4096-token context • 2048-token output

Objects

plate • cup • fork • apple container

Action space

pick • place • move_home

Two scenario families

- Direct instruction following — “clear the table”, no disturbance
- Execution-time disturbances — object moved, grasp failure, blocked object

120

trials in total

12 scenario definitions

× 5 repetitions

× 2 methods

What is compared, and how it is measured

LLM-only planning (baseline)

Receives a simplified textual summary of the simulator state and generates the complete symbolic action sequence.

Ontology-grounded planning (proposed)

Receives table-related objects with their ontology-derived storage targets and returns an ordered sequence of grounded object–target transfers.

Same LLM backend • same validator • same robot skills • same replanning mechanism

Feasibility

Trials whose initial symbolic plan is valid under the current scene constraints

$$N_f / N \times 100$$

Success

Trials in which the robot reaches the intended final task state

$$N_s / N \times 100$$

Replanning

Disturbed trials in which the system recovers and completes the task

$$N_r / N_d \times 100$$

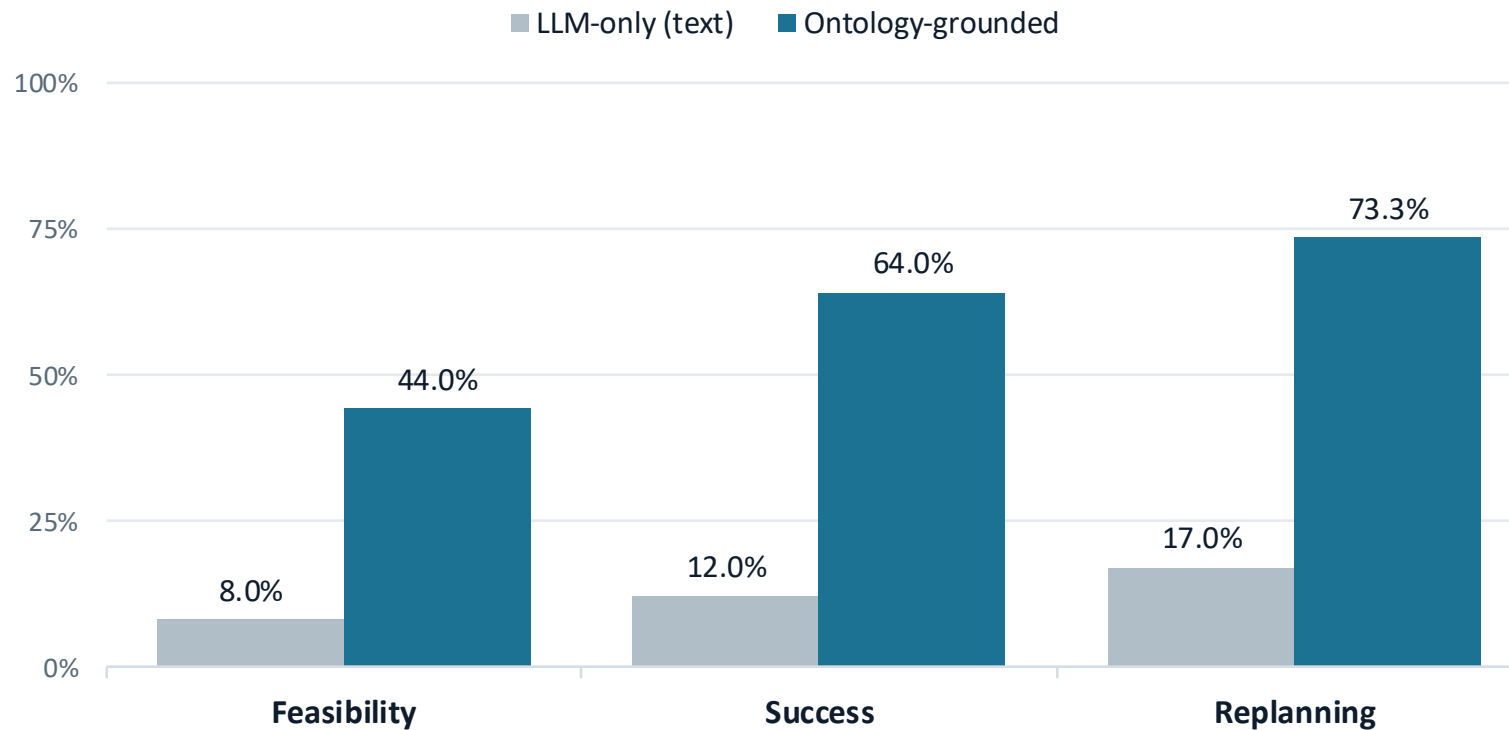
Planning time

Mean time to generate a valid symbolic plan across all evaluated trials

$$(1/N) \sum T_i$$

RESULTS

Grounding improves feasibility, success and recovery



INITIAL PLANNING TIME

~~65.5~~ → **15.2 s**

≈ 77% faster to a valid plan

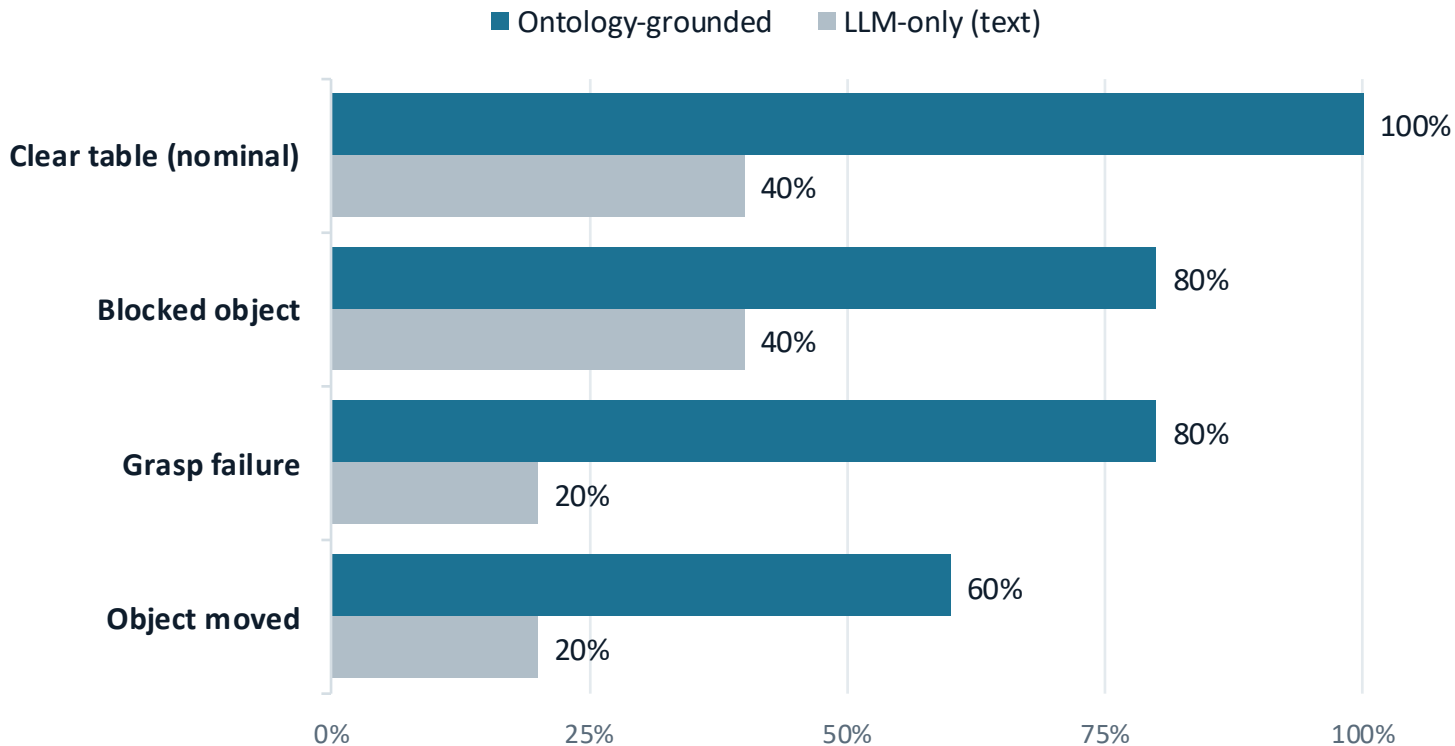
Feasibility **8.0% → 44.0%**

Task success **12.0% → 64.0%**

Replanning success **17.0% → 73.3%**

Aggregate over the evaluated trial set (Table III). Planning-time gain should be read at the pipeline level: the two configurations use different output formats.

The gap widens exactly where the scene changes



Five trials per scenario, per method (Table IV).

Nominal clearing

5 / 5 trials completed by the proposed method; the text-only baseline reached 40%.

Blocked object

Support relations tell the planner what must be removed first — 80% vs 40%.

Grasp failure

Reconstructing the state after a failed grasp recovers the task — 80% vs 20%.

Object moved

Hardest case: the planner must reinterpret a scene that changed spatially — 60% vs 20%.

What this shows — and what it does not

What the results support

Tight coupling is what matters

Keeping the planner tied to a world model that is updated during execution improves both initial plan consistency and recovery.

Recovery without extra prompting

Replanning is driven by state reconstruction from ontology and simulator, not by telling the LLM that something went wrong.

Limitations we state openly

Feasibility is still only 44%

The framework continues to fail in a considerable number of trials, before or during execution.

Spatial change remains hardest

Object-moved disturbances stay below blocked-object and grasp-failure recovery.

Narrow evaluation scope

One household task family; ambiguous instructions and mid-task correction are not systematically evaluated.

CONCLUSION

The reliability of LLM-based task planning depends on how tightly planning is connected to world-state representation and execution feedback.

Plans from a grounded scene

Not from free text: ontology semantics merged with simulator geometry.

Validated before execution

Symbolic checks and bounded repair keep infeasible plans off the robot.

Revised during execution

State reconstruction and replanning turn a failed skill into a recoverable step.

Next →

Richer interaction settings and more diverse household manipulation tasks · a systematic comparison between VLM-based grounding and the symbolic ontology grounding used here.

Thank you

Questions are welcome.

Ontology-Grounded LLM Planning for Robust Table-Clearing Robot Manipulation

Satuk Buğrahan ÖZTÜRK · Ahmet COŞKUNÇAY · Erhan ÖZTOP · Barış ÖZYER

Supported by the Atatürk University Scientific Research Projects Coordination Unit (BAP), reference number FDK-2024-14650.